Transcript

Week 7: Supervised Learning- Classification- Part 1

Video 1 - Supervised Learning

How can machines learn to recognise patterns, make decisions, and even predict the future—all without being explicitly programmed?

A machine learning approach starts with a model trained on labeled data. The core idea of this method is to learn a mapping from input features to output labels using the given training data. During this process, the model learns from examples where each input is paired with a corresponding label. The objective is to understand the relationship between inputs and labels so the model can predict labels for new, unseen inputs.

Through iterative training, the model fine-tunes its internal parameters to reduce the error between its predictions and the actual labels. This iterative process allows the model to generalise and make accurate predictions on new data. By mastering this mapping, machine learning models deliver valuable insights, automate decision-making, and enable intelligent solutions across various applications.

Types of supervised learning include regression and classification. Regression is used when the output is a continuous value, such as predicting a house's price based on its features. Classification is used when the output is a discrete label or category, such as identifying whether an email is spam. Both methods rely on labeled data to train the model and make accurate predictions.

Learn more about regression and classification by exploring simple examples in everyday scenarios.

For learner use only

Video 2 - Classification

Can you think of a real-world scenario where classification could help, like categorising customer feedback or detecting fraud? Try applying it to your own examples!

Classification is a supervised learning method aimed at assigning inputs to predefined categories or classes. The goal is to predict the category of input based on its features. In classification, the model learns from labeled examples to classify new, unseen data into one of the specified classes.

For instance, in email spam detection, the model classifies emails as 'spam' or 'not spam' based on their content. In image classification, it identifies objects within images, such as categorizing them as 'cat,' 'dog,' or 'bird.' Similarly, in fraudulent transaction detection, the model classifies transactions as 'fraudulent' or 'legitimate' based on transaction patterns. These examples illustrate how classification predicts the category an input belongs to using its features.

In the training phase of classification, the initial step is data preparation. This involves collecting and labeling data with known categories, which serves as the foundation for learning. During model training, algorithms analyse the labeled data to understand the relationship between features and class labels.

In the prediction phase, new, unlabeled data is fed into the trained model. The model classifies this data by assigning the most likely class label based on its training.

Classification algorithms are widely applied across various fields.

In medical diagnosis, they identify diseases based on patient symptoms, enabling early detection and targeted treatment. Spam detection uses classification to filter spam emails from legitimate ones, keeping inboxes organized and relevant. Image recognition classifies objects within images, such as distinguishing between cats and dogs, supporting applications like photo tagging and autonomous vehicles. Speech recognition converts spoken words into text, enabling features like voice commands and automated transcription services.

These applications highlight the versatility and impact of classification across domains.

Binary classification is the process of assigning data to one of two possible classes, aiming to determine whether an instance belongs to class A or class B. Several algorithms are commonly used for this task. Logistic regression models the probability of a binary outcome, while support vector machines identify the optimal hyperplane that best separates the two classes. Naive Bayes, on the other hand, relies on probability theory and assumes feature independence to make classifications.

Binary classification has a wide range of applications. In spam detection, it helps distinguish between spam and non-spam emails. In medical diagnosis, it is used to identify the presence or absence of diseases, such as cancer. Similarly, in fraud detection, it classifies transactions as either fraudulent or legitimate, ensuring better security and risk management.

Multi-class classification is the process of assigning data to one of three or more possible classes, aiming to determine the correct category for each instance. Several algorithms are commonly used for this task. Decision trees split data into branches based on feature values, allowing for structured decision-making. The k-nearest neighbors algorithm classifies instances by identifying the majority class among the nearest neighbors. Neural networks, with their multiple layers, learn complex patterns to accurately categorize data into multiple classes, making them highly effective for diverse classification problems.

Multi-class classification has a wide range of applications. In handwriting recognition, it classifies digits into categories from 0 to 9. Image categorization enables the identification and labeling of different objects within images. Similarly, in language identification, it determines the language of a given text, making it useful for translation and communication tools.

Multi-label classification assigns multiple labels to each instance rather than a single class, aiming to predict a set of relevant labels for each data point. Several algorithms are used for this task. Binary relevance treats each label as a separate binary classification problem, allowing independent predictions. Classifier chains build a sequence of classifiers, considering previous label predictions to enhance accuracy. Neural networks, with multiple output nodes, can predict several labels simultaneously, making them highly effective for complex multi-label tasks.

Multi-label classification has various real-world applications. In image tagging, it assigns multiple tags such as "beach," "sunset," and "vacation" to a single photo. In text classification, it categorizes an article under multiple topics like "politics," "economy," and

For learner use only

"technology." Similarly, in music genre classification, it labels a song with multiple genres, such as "rock," "pop," and "indie," allowing for more accurate and flexible categorization.

Hierarchical classification organises data within a multi-level structure, assigning instances to categories that may include subcategories. This system aims to classify data accurately within the hierarchy. Specialised algorithms support this process, such as decision trees with hierarchical structures, which create tree-like models for multi-level categorisation, and hierarchical SVM, which adapts Support Vector Machines to handle nested relationships while maintaining the hierarchy.

Hierarchical classification is commonly used to organise complex data structures effectively. For example, in document classification, documents are first grouped into broad categories like "Science" and then refined into subcategories such as "Physics" or "Biology." Similarly, in e-commerce, products are categorised hierarchically, such as "Electronics" → "Laptops" → "Gaming Laptops," making organisation clearer and search functionality more efficient.

Ordinal classification assigns data to ordered categories where the order is important, but the exact distance between categories is undefined. The goal is to predict the rank or order of instances within predefined categories. Algorithms like ordinal regression classify data into these ordered categories without defining exact distances, while ordered logistic regression ensures classifications maintain the natural order, making it well-suited for ranking-based predictions.

Ordinal classification has practical applications in various fields. For instance, in customer satisfaction surveys, it is used to rate experiences on an ordered scale, such as "very dissatisfied" to "very satisfied." Similarly, in academic grading, it ranks student performance using letter grades like A, B, C, D, and F, reflecting an ordered hierarchy of achievement.

The purpose of evaluating a classification model is to determine how effectively it predicts class labels for new, unseen data. This process is vital for understanding the model's accuracy and reliability in making predictions.

Evaluating a classification model's performance is essential to ensure it generalizes well beyond the training data. By analyzing metrics like accuracy, precision, recall, and F1-score, we can assess the model's performance on unseen data. This evaluation helps confirm whether the model is likely to make accurate predictions in real-world scenarios, validating its practical reliability and usefulness.

A confusion matrix is a table used to summarise the performance of a classification model by displaying the counts of true positives, true negatives, false positives, and false negatives. True positives represent instances correctly predicted as belonging to the positive class, while true negatives are instances correctly predicted as part of the negative class. False positives occur when instances are incorrectly predicted as positive despite being negative, and false negatives arise when instances are incorrectly predicted as negative despite being positive. By presenting this information, the confusion matrix not only evaluates the overall accuracy of the model but also assesses its ability to distinguish effectively between positive and negative classes.

For learner use only

Accuracy and precision are key metrics for evaluating the performance of a classification model. Accuracy measures the proportion of correctly predicted instances, including both

true positives and true negatives, out of the total instances in the dataset. It provides an overall assessment of the model's correctness.

On the other hand, precision focuses on the proportion of true positive predictions among all instances predicted as positive. It indicates how many of the predicted positive cases are actually correct, offering insights into the reliability of the model's positive predictions.

Recall or Sensitivity measures the proportion of correctly identified positive cases among all actual positive instances. It reflects the model's ability to detect positive cases effectively. Higher recall means fewer false negatives.

F1 Score is the harmonic mean of precision and recall, providing a balanced metric when dealing with imbalanced datasets. It ensures that both precision and recall are considered, making it useful when a trade-off between the two is necessary.

Classification tasks can be tackled using various algorithms, each employing a unique approach to predict class labels. Logistic regression models the probability of a binary outcome, predicting which of two classes an input is most likely to belong to. Decision trees create a tree-like structure by splitting data based on feature values, making decisions at each node to classify inputs. Random forests enhance accuracy by combining predictions from multiple decision trees, improving overall performance. Support Vector Machines identify the hyperplane that best separates classes, maximising the margin between them. k-nearest neighbors classifies inputs based on the majority class among their nearest neighbours in the feature space. Neural networks, with multiple layers of interconnected nodes, learn complex patterns, making them effective for handling intricate data relationships. Each algorithm has unique strengths, making it suitable for different classification problems depending on the data and task requirements.

Explore the strengths of different classification algorithms by experimenting with them on various datasets and understanding how each one performs based on the data characteristics.

For learner use only

Video 3 - Logistic Regression

Imagine you're working on a project to predict whether a patient has a certain disease based on medical test results. How could logistic regression help in this binary classification task, and what factors would you include as input features?

Logistic Regression is a regression algorithm designed for binary classification problems. Its objective is to predict the probability of an instance belonging to one of two classes, usually labeled as 0 and 1.

Unlike linear regression, which predicts continuous values, logistic regression models the probability of an input belonging to a specific class. It uses the logistic function to map predicted values to probabilities, ensuring outputs remain between 0 and 1.

Logistic regression is commonly applied in tasks like spam detection, medical diagnosis, and other binary classification problems.

Logistic regression uses the sigmoid function to map predicted values to a probability between 0 and 1. The sigmoid function takes a linear combination of input features and

transforms it into a probability value. This ensures that the output of the model is always within the valid probability range.

In logistic regression, the decision boundary is established by comparing the predicted probability to a set threshold, typically 0.5. If the probability of belonging to the positive class is at least 0.5, the model assigns the input to that class. Otherwise, it classifies the input into the negative class. This boundary allows the model to effectively differentiate between two categories in binary classification tasks.

The sigmoid function in logistic regression transforms a linear combination of features into a value between 0 and 1, representing the predicted probability. The linear part combines the input features with their respective weights, which are learned by the model during training. These weights determine the influence of each feature on the prediction. The sigmoid function ensures the output is always a valid probability, making it suitable for binary classification tasks. For instance, if the linear combination results in a value of 2, the sigmoid function predicts an 88% chance that the output belongs to the positive class. This transformation is key to how logistic regression models make predictions.

In logistic regression, the coefficients or weights of the model are determined using maximum likelihood estimation. This method identifies the values of the coefficients that maximize the likelihood of the predicted probabilities matching the actual labels in the dataset.

The process involves calculating the likelihood function, which measures how well the predicted probabilities align with the true outcomes. To simplify the calculations, the log-likelihood is often used instead of the likelihood function. The log-likelihood transforms the multiplication of probabilities across data points into additions, making the optimization process more efficient and manageable.

The log-likelihood formula for binary classification measures how well a model predicts the outcomes for a dataset. It evaluates the probability that the predictions align with the actual labels . If the actual label is 1, the goal is for the predicted probability to be as close to 1 as possible. If the actual label is 0, the predicted probability should be as close to 0 as possible. The formula combines these probabilities for all data points to calculate the overall log-likelihood.

Logistic regression works like predicting the outcome of a coin flip. It learns from past flips to estimate the probability of landing on heads or tails. If the actual outcome is heads, the model should predict a probability close to 1, and if it's tails, the probability should be near

0. The log-likelihood function measures how well these predictions match real outcomes. A higher log-likelihood means better predictions, while a lower value indicates poor guesses.

Gradient Descent is the optimisation algorithm used in logistic regression to determine the values of beta that maximise the log-likelihood, ensuring the model accurately fits the data. It works by iteratively adjusting the model's coefficients in the direction that

For learner use only

increases the log-likelihood. Gradients, which represent the slope or direction of change, guide these adjustments. The size of each update is controlled by the learning rate alpha. A smaller learning rate ensures more precise but slower updates, while a larger learning rate accelerates the process but risks overshooting the optimal values. This iterative approach is crucial for fine-tuning the model.

Logistic regression has several advantages that make it a popular choice for binary classification tasks.

Logistic Regression is valued for its simplicity, as it is easy to implement and interpret, offering clear insights into how features influence predictions. It is highly efficient, particularly with linearly separable data and small to medium-sized datasets, making it well-suited for straightforward problems. Additionally, it provides probabilistic outputs, allowing for flexible decision-making and the adjustment of thresholds based on specific needs or desired confidence levels.

Despite its strengths, logistic regression has several limitations:

Logistic Regression operates under the linearity assumption, presuming a linear relationship between input features and the log-odds of the outcome, which may not always be accurate. It is limited to handling simple relationships and struggles with non-linear patterns unless features are transformed or advanced techniques are applied, making it less effective for complex datasets. Additionally, while it is primarily designed for binary classification, it can be extended to multi-class problems through methods such as softmax regression.

Logistic regression is widely applied in various real-world scenarios:

Logistic Regression is widely used in various applications, such as medical diagnostics, where it predicts the likelihood of a patient having a disease based on symptoms and medical data, aiding in early detection and treatment. In customer churn prediction, it identifies whether a customer is likely to leave a service, enabling companies to implement effective retention strategies. Additionally, it is employed in email spam detection, classifying emails as spam or not, helping to keep users' inboxes free f rom unwanted messages.

Now, how can you apply logistic regression to solve challenges in your field? Think about it!

For learner use only

Video 4 - Logistic Regression Implementation

What libraries would you import to handle data manipulation, numerical operations, and model evaluation when starting a machine learning project, and why are they important?

To start, we import the necessary libraries for data manipulation, numerical operations, and model evaluation

We then load the Titanic dataset and perform initial data preparation.

This code processes the Titanic dataset by loading it into a pandas DataFrame and displaying its basic structure. It fills missing values in the Age column with the median and replaces missing values in the Embarked column with the most common value.

Categorical data, such as Sex and Embarked, is converted into numeric form. Finally, unnecessary columns like Name, Ticket, and Cabin are removed, and the cleaned dataset is displayed.

We prepare the data for training and testing then We initialise and train the logistic regression model.

This code snippet shows how to create and train a Logistic Regression model. The model is first initialized with a maximum iteration limit of 1000 to ensure it has enough cycles to reach convergence. It is then trained using the fit method with the training data, where input features and target labels are provided. After training, the model's configuration, including the maximum iterations, is displayed.

Finally, we make predictions and evaluate the model's performance on train data.

This code assesses a model's performance on training data by predicting labels and comparing them to the actual ones. It generates a classification report with key metrics such as precision and recall. Additionally, a confusion matrix is calculated to summarise true and false positives and negatives. This matrix is then visualised as a labelled heatmap, using annotations and a blue colour scheme for better clarity.

We also make predictions and evaluate the model's performance on test data.

This code evaluates the model's performance on test data by predicting labels, generating performance metrics, and visualizing the confusion matrix as a heatmap for better interpretation.

The confusion matrices for both the training and test datasets provide insights into the Logistic Regression model's performance. The model achieves an accuracy of 80% on both the training and test datasets, indicating consistent performance across different data splits.

The model shows consistent accuracy on both the training and test datasets, suggesting it has effectively learned the underlying patterns in the data without significant overfitting.

The precision for Class 0 is slightly higher than for Class 1 in both the training and test datasets, indicating that the model is more accurate in predicting non-survivors. However, the precision for Class 1 is still reasonably high, showing the model's effectiveness in identifying survivors. The recall for Class 0 is higher than for Class 1 in both datasets, reflecting the model's stronger performance in identifying non-survivors while still effectively detecting survivors. The F1-score, which balances precision and recall, remains consistent across the training and test sets, demonstrating that the model achieves a good trade-off between precision and recall for both classes.

The observation indicates no significant evidence of overfitting, as the accuracy, precision, recall, and F1-score metrics for both the training and test datasets are consistent. This similarity suggests that the model generalises well to unseen data, as overfitting would

For learner use only

typically result in substantially higher training metrics compared to test metrics, which is not the case here. The Logistic Regression model demonstrates strong performance with consistent metrics across both datasets. Its ability to accurately identify non-survivors while maintaining reasonable performance in identifying survivors highlights its balanced and effective learning.

Now, think about, how you can ensure your model generalizes effectively without overfitting.

Video 5 - k-Nearest Neighbors (k-NN)

How could you use k-Nearest Neighbors to predict the price of a house based on the features of nearby houses, and what distance metric would you choose for this task?

k-Nearest Neighbors is a versatile algorithm used for both classification and regression. It makes predictions by finding the closest data points to a given instance and determining the outcome based on their majority class or average value. In classification, it assigns the most common class among the nearest neighbors, while in regression, it predicts by averaging their values. k-NN identifies these neighbors using distance metrics like Euclidean or Manhattan distance, making it an effective method for pattern recognition.

The k-NN algorithm is versatile, handling both classification and regression tasks by relying on the proximity of data points. In classification, it identifies the k closest neighbours to a given instance and assigns the most common class label among them, effectively using a majority vote to classify the instance. In regression, the algorithm also identifies the k nearest neighbours but predicts the value by averaging the values of these neighbours, offering a smoothed estimate based on their proximity. This approach ensures flexibility in addressing various types of problems.

The goal of classification is to determine the class of an instance based on its nearest neighbors. For example, if the three closest neighbors belong to classes A, A, and B, the instance is assigned to class A since it appears most frequently among the neighbors.

In regression, the goal is to predict the value of an instance by averaging the values of its nearest neighbors. For example, if the five closest neighbors have values of 10, 12, 11, 13, and 10, the predicted value would be their average, which is 11.2. This approach provides a smooth estimate based on nearby data points.

k-NN uses different distance metrics to identify the closest neighbors. Here are two common ones:

Euclidean Distance measures the straight-line distance between two points by calculating the square root of the sum of the squared differences between their corresponding features, representing the shortest path between the points in the feature space. Manhattan Distance, on the other hand, calculates the distance by summing the absolute differences along each dimension, similar to navigating grid lines by moving only horizontally and vertically, measuring the total distance traveled.

The Minkowski distance is a flexible metric that can represent both Euclidean and Manhattan distances. Its adaptability comes from a parameter, p that determines the calculation method: when p = 2, it functions as Euclidean distance, and when p = 1, it behaves like Manhattan distance. By adjusting p, this distance measure can be tailored to

For learner use only

various scenarios, making it a powerful tool for analyzing spatial relationships in feature space.

To classify a fruit as an apple or an orange based on weight and color, consider the given data: Apples weigh 150 grams and 160 grams and are red, while oranges weigh 200 grams and 210 grams and are orange. If a new fruit weighs 170 grams and is red, using k = 3, the three closest neighbors are the apples weighing 150 grams and 160 grams, and the orange weighing 200 grams. Since the majority of the nearest neighbors are apples, the fruit is classified as an apple.

The k-NN algorithm has its advantages and limitations. One of its key benefits is its simplicity, making it easy to understand and implement. Additionally, it does not require a separate training phase, allowing the model to be updated with new data without retraining. However, k-NN can be computationally expensive, as it searches through the entire dataset for every prediction, which can become a bottleneck with large datasets. It is also sensitive to feature scaling, as features with larger scales can disproportionately influence distance calculations, potentially affecting results. Finally, the algorithm's performance heavily depends on the choice of k or the number of neighbours and the distance metric used, as inappropriate choices can negatively impact prediction

accuracy.

Now, what are your thoughts on using k-NN, given its simplicity and flexibility, as well as its computational and scaling challenges?

Take a moment to reflect!

For learner use only

Video 6 - KNN Implementation

What libraries would you import first when starting a machine learning project to handle data manipulation, numerical operations, and model evaluation, and why are each of these important?

Next, we load the Titanic dataset and perform initial data preparation.

The code loads the Titanic dataset, fills missing values, converts categorical columns to numeric, drops unnecessary columns, and displays the cleaned data.

We prepare the data for training and testing, then initialize and train the logistic regression model.

The code sets up a K-Nearest Neighbors classifier with three neighbors and trains it using the provided training dataset, which includes input features and target labels.

This code assesses the accuracy of a K-Nearest Neighbors model on training and test data for different numbers of neighbors, ranging from 1 to 29. It records the accuracies, plots them against the number of neighbors, and visualizes the comparison using a line graph to analyze training and test performance.

The graph shows the training and test accuracy of the KNN model for different values of neighbors. It highlights the model's performance trends, with training accuracy decreasing and test accuracy stabilizing as the number of neighbors increases.

The plot compares training and test accuracy of the KNN model as the number of neighbors increases from 1 to 29, showing how the model's performance varies.

Based on the accuracy analysis, K = 10 is selected as the optimal number of neighbors to prevent overfitting. A K-Nearest Neighbors model is then created with 10 neighbors and trained using the provided training dataset.

Finally, we make predictions and evaluate the model's performance on the training data.

This code evaluates the model on training data by generating predictions, calculating performance metrics, and visualizing the confusion matrix as a labeled heatmap.

We also make predictions and evaluate the model's performance on test data.

From the confusion matrices for the training and test datasets, the following insights into the performance of the Logistic Regression model can be derived:

The overall accuracy of the model is 69% on the training dataset and 68% on the test dataset.

The model demonstrates consistent accuracy on both the training and test datasets, indicating it has effectively learned the underlying patterns in the data without significant overfitting.

The Logistic Regression model's performance, based on the confusion matrices for both the training and test datasets, provides valuable insights. The model achieves an overall accuracy of 69% on the training dataset and 68% on the test dataset, indicating consistent performance and effective learning of data patterns without significant overfitting.

For the training dataset, the model performs well in identifying Class 0 , with a precision of 0.69, a high recall of 0.90, and an F1-score of 0.78, reflecting a strong balance between precision and recall. However, for Class 1, while precision remains at 0.69, recall drops to 0.36, leading to a lower F1-score of 0.47, showing difficulty in accurately identifying positive instances.

On the test dataset, the model's performance for Class 0 is consistent, achieving a precision of 0.69, recall of 0.89, and an F1-score of 0.78, ensuring reliable identification of non-survivors. However, for Class 1, precision decreases to 0.63, and recall further drops to 0.33, resulting in an F1-score of 0.43, highlighting challenges in accurately predicting survivors on unseen data.

Overall, while the model reliably identifies non-survivors, it struggles with accurately predicting survivors, particularly in terms of recall and F1-scores.

The model achieves 69% accuracy on the training dataset and 68% accuracy on the test dataset, showing consistency. This suggests that the model is not significantly overfitting, as overfitting typically leads to much higher training accuracy compared to test accuracy. The similar results indicate that the model generalizes well to new, unseen data.

The class-specific performance of the Logistic Regression model reveals stable results for Class 0, with consistent precision, recall, and F1-scores across both the training and test datasets. This indicates that the model performs reliably for this class without signs of overfitting. However, for Class 1, there is a noticeable drop in recall and F1-score on the test

For learner use only

dataset compared to the training dataset. This suggests that the model may have learned patterns specific to the training data, which do not generalize well to unseen data, indicating some degree of overfitting for Class 1.

What steps could we take to improve Class 1 performance and reduce overfitting? Pause and reflect!

Video 7 – Model Evaluation

Welcome to this session. We will explore how to evaluate classification models.

Model evaluation ensures that our model doesn't just perform well on training data but generalises effectively to unseen data. Choosing the right evaluation metric is essential and depends on the context of the problem like whether we are dealing with class imbalance or high stakes decisions. We'll look at a wide range of metrics each offering a different lens to assess performance.

One thing I want to tell you here when we talk about imbalanced data, imbalanced data is such data set where you have too many examples for one of the class but very less number of examples for another class. That's called imbalanced data. Ideally, we want 50 percent data point for one class and 50 percent for other class if I have binary classification.

So that's called class imbalance and the problem what happens with the imbalanced classes is that it makes your model biased towards the class which has majority of the examples. Now here we start with the confusion matrix. Most classification metrics stem from the confusion matrix so it's fundamental to understand.

We are taking example of binary classification. It's a two-by-two table with true positive true negatives just to let you know what it means. So when we have the data and I'm doing binary classification let's say we are talking about the diabetes data set.

We want to predict if somebody is having diabetes or not. So when my model is predicting it can say the person is having diabetes let's say class one and person is not having diabetes as class zero and that's my model prediction. So this over here is your predicted outcome and this side we are taking what's the actual label given.

So for the same person what's the actual label was that person actually in actual they were if that person was diabetic or not and then we go and check how many times it's happening that my model is saying positive and the actual label is also positive. So we go through the entire data set and we check whenever model is saying one and if actual label is also one then count that and write over here that's called true positive. Similarly you count that number when the model is saying zero and the actual label is also zero and that's called true negative.

Now true positive and true negative is basically they are your correct predictions. So these are your correct predictions but then also what happens is there might be a scenario where your model is saying zero but the actual label is one. So from moral perspective that's a false negative.

Similarly there might be a scenario where your model is saying one but the actual label is zero and that's called false positive. So this is the this is called confusion matrix and when

For learner use only

you do it in python the confusion matrix will come like this: class zero class one as per the model and class zero and class one as per the actual label. And you'll get true negative here true positive over here this is false positive and this is false negative. That's your get to see and ideally false positive and false negative should be zero but in actual scenario you will have some points for false positive false negative, as long as they are relatively very very small compared to true positive and true negative, we are happy about it. Else our model is

not so good and when you want to implement this - it's very simple in sklearn you can import confusion matrix from sklearn.matrix provide the actual label and predicted label it will create the confusion matrix for you. Now based on confusion matrix we calculate something called accuracy.

Accuracy is the simplest and most commonly used metric. It measures the proportion of correctly predicted instances both positive and negatives out of all predictions. So simply counts how many true positives are there how many true negatives are there, basically a correct prediction divided by total number of observation, which is nothing but sum of true positive true negative false positive and false negative. It ranges from zero to one and if you want to implement you can pick up a function called accuracy score from sklearn metric and provide the actual label and predicted label it will tell you the accuracy. Ranges from zero to one-one means perfect prediction zero means totally random prediction. You have to remember one thing you have to be very cautious in cases of class imbalance. And accuracy can be misleading, like for example suppose, I have a data set where I have regular transactions-I'm marking as class zero and I have fraudulent transaction-I'm marking as one and I have 9900 transactions which are regular and 100 of them are fraudulent because number of fraudulent transaction will be extremely small given the number of transaction that happens every month. Now this is just a hypothetical situation so I'm talking about here I'm saying let's say, a model is built on this data with which had 10 000 data points, and 9900 of them belong to class zero. Now what happens is if I get this model, which can classify fraudulent transaction perfectly then good but generally what happens that they get biassed towards regular transaction because it has more examples. So let's say there is a model which classifies everything as regular even the fraudulent one as regular so all these 9900 will be classified as regular class zero and all these 100 will also be classified as regular as class zero. So these are wrong prediction so how many correct predictions my model has done 9900 total number of observation 10 000. What's the prediction accuracy 99 but my model is good for nothing it has not identified a single fraudulent transaction so my accuracy could be misleading if you have imbalanced data set and that's the reason; we need a little more than accuracy. So one of the metrics that is very useful is called precision. Precision focusses on the quality of positive predictions it answers of all the instances predicted as positive how many were actually positive. It's especially useful when the cost of a false positive is high, like if the email is spam or not and precision ensures that the positive predictions we make are trustworthy. How do we calculate it it's given as true positive divided by true positive plus false positive so find number of observations for true positive and false positive. Calculate this ratio and then you will get precision value, like in like in our example of fraudulent transaction true positive was zero and false positive are also zero, I'll get a value as zero so my precision is zero in my previous example, although accuracy was 99 percent, which tells me I fail to predict any of the fraudulent transaction and again if you want to calculate precision you can import it from sklearn metrics and then you can check the precision score.

For learner use only

Next, we have recall. Recall measures the ability of a model to capture all actual positives. It asks of all the true positives how many did we identify. This is crucial in high-risk applications like disease detection we're missing a positive case a false negative could have severe consequences. A model with high recall ensures fewer misses - like even in our example if a transaction is fraudulent or not, if I miss to correctly classify a fraudulent transaction then that will come as a false negative and that's a big challenge. Now when you want to calculate recall you take the true positive number of true positive cases divided by true positive cases Guwahati

plus false negative. Again the range is from zero to one and if you want to implement you can use a scale and metrics recall score. Then you have f1 score-the f1 score is the harmonic mean of precision and recall, offering a balance between the two. It's a useful metric when we want a single value that reflects both aspects, especially when dealing with imbalanced data sets. The range is zero to one a high f1 score indicates that the model performs well in identifying positive while minimising both false positive and false negatives. Then we have specificity-focusses on the true negatives and answers how well does the model identify the negative class. It complements recall while recall tells us about sensitivity to positives specificity tells us how well the model avoids false alarms. This is key in scenarios where false positive are costly like fraud detection. We do not have a function in a skill learn metric to get specificity but we can define our own function it is given as true negative divided by true negative plus false positive. We can get the confusion matrix from there we can get true negative false positive false negative and true positive and then return the specificity value.

Then you have ROC curve. ROC curve works on the premises of some of the probabilistic model. Generally what we do, when we build the model, we are modelling this what's the probability of y is equal to one that's what we are modelling and generally say if this probability is more than or equal to 0.5 then classify it as one else classify it as zero. So we take this cut off and for this cut off we calculate the true positive rate, which is nothing but the recall value, then the false positive rate, which is false positive divided by false positive plus two negative. So this will give you the TPR value and the FPR value for this cut off. And then what we do we change this cut off we try different cut off, because changing a cut off will change the model's prediction and it might actually get better if you change the cut off. So what we are doing over here is we're going to try a threshold of 0.1, 0.2, 0.3 till one, and then for each of these thresholds we'll calculate what's the true positive rate and what's the false positive rate. And once we calculate that then we will plot this true positive rate and false positive rate at different thresholds, that's what we're going to do. And once we do this, we'll get this kind of curve. Now if your cut off of 0.5 is giving you the best result then this curve will come like this else, you'll have to check for some other cut off.

So the different threshold you get different TPR and FPR. You are looking for such value where TPR is maximised and FPR is minimised. So looking at the ROC curve you try to understand where is my TPR maximised? My TPR is maximised or one of the value I can think here although the TPR is maximum here and here but that also increases the false positive rate so I want to strike a balance somewhere here if I choose my TPR is over here and my false positive rate is over here. I can say okay this looks fine so what is the threshold for this value that's the threshold I will use to classify my data points. So ROC curve gives you good understanding what kind of cutoff we can use but this is used very

for learner use only

in a very special cases where your original model is not working well after doing everything. So you try to change the cutoffs now based on the ROC curve we also have something called AUC. AUC or area under the ROC curve condenses the ROC curve into a single number between 0.5 and 1. A score of 0.5 indicates random performance while one represents a perfect classifier it's a preferred metric for comparing classifiers using probability scores rather than hard predictions. Then you have precision recall curve. The precision recall curve is especially valuable when dealing with imbalanced data sets. It shows the trade-off between precision and recall at various thresholds. Unlike ROC which considers all predictions, precision curves focus only on the positive class, making them more informative in certain contexts, like rare event predictions. Again we are looking for such threshold where my precision and recall both are maximised. In this example okay so we are looking for such threshold where my

precision is also high and recall is also high, so in a perfect scenario curve will come like this and my threshold whatever the threshold for this particular value but here you have different precision, different recall.

Another metric is log loss which penalises the false confident predictions by comparing predicted probabilities to actual outcomes. So this focusses on comparing the actual probability with the predicted probability. A model that assigns high confidence to wrong predictions will have a higher log loss. It's ideal for probabilistic classifier where we want to not just correct labels but calibrate probability. Estimates ranges from zero to infinity. High log loss means you are doing wrong prediction.

Then you have Matthew's correlation coefficient. It provides a balance metric that considers all four quadrants of the confusion matrix, especially valuable when classes are imbalanced. A perfect prediction gives plus one random prediction zero and total disagreement minus one. MCC is often used in bioinformatics and other high stakes field due to its robustness. Again you can use a scale and metrics to get this value.

Then you have Cohen's kappa score. Cohen's kappa measures the agreement between predicted and actual labels without while accounting for agreement that happens by chance. A score of one indicates perfect agreement, zero suggest random agreement and minus one means total disagreement. It's commonly used in scenarios like medical diagnosis and human annotation tasks.

Another metric we have is your balance accuracy, calculates the average recall across all classes. So for each class calculate the recall. They can take the average of that - that's your balance accuracy. And this makes it useful for imbalanced data set where standard accuracy might be misleading. It ensures both classes are equally represented in performance evaluation providing a more fair assessment. Again I can use a scale and metrics for it.

Now here's a quick summary of metrics provides this table provides a summary of all metrics discussed. It's essential to choose the metric that aligns with your use case. For example, a medical diagnosis recall might matter more while in spam detection. Precision might be key for imbalanced data sets. Metrics like f1 MCC or and PR AUC offer better insights and accuracy alone.

Now let's do one thing, let's take an example and see how it's getting implemented in real-time scenario. So here we are taking one example of diabetic data set. We have the

For learner use only

diabetes data and we have loaded the data set, we have loaded all the libraries, then we have loaded the diabetes data set x and y. And if you look at the detail of the data set this has 442 observations. First 10 columns are numeric predictive value, and then column 11 is quantitative measure of disease progression one year from baseline, so we have these kind of attributes - age in years, sex is given, BMI-body mass index and so on. And then you have a target variable 0, 1-whether person is diabetic or not.

So we are building a model here. We are defining model logistic regression and we are fitting the model. And here we are getting what's a train prediction. So model.predict on x_train, y_test prediction model. Predict x_test and we are also getting the probability scores. What's the train probabilities and what's the test probabilities, what's the probability. And here

remember we are modelling what's the probability of y is equal to 1, so this probability here they represent what's the probability of y is equal to 1.

And then we are defining a function to calculate the specificity, which was, that doesn't come from the metric. Rest of the things will come from the metric, so we are defining evaluate metrics-that's a function which takes the actual label the predicted label. And the probability value for y_prob that we are talking about, so that's the probability of y is equal to one label is equal to set. We are simply providing a label and then we are printing here label metric like train or test. What's the accuracy precision recall, f1 score, specificity, ROC, log loss, MCC, Cohens, combined balanced accuracy. And then we are applying this function on y_ train-that's actual label for the train data y_train. Predicted is a predicted label y_train. Probability and the set name is train and same thing we are doing for the test.

And here is the accuracy of my model. Overall accuracy is 74 which comes on the test is 77 which gives us the idea model is quite good. It's underfitted, but when I look at the balance accuracy, I'll get a good understanding. When balance accuracy is same, so my model is not having overfitting problem it has a bit of underfitting problem. Precision is 0.75 on train 0.74 on test which is quite comparable. Recall also is quite comparable on both so we can say model is not overfitted most of the values are between 0.7 to 0.8 which is fine.

Okay, now here we're going to plot ROC curve, so I want false positive rate and true positive rate. I can use a function called ROC curve underscore. Over here is actually it also gives you the threshold which I am not storing and then I provide y_test y_test probability. And this is how the ROC curve is coming which is telling me it's not a perfect scenario because in the perfect scenario I could have got this kind of graph. So it tells us that there is a scope over here where we can change the threshold and we can get even better model and then we are also doing the precision recall curve. Calculate the precision recall again threshold we are not storing, and then plot the precision recall curve which comes like this again. In an actual in an ideal situation it should come like this, not in actual but ideal situation it should come like this, so model is average model. I would say it's not that great it's not that poor either so this is how you can get model predictions and different accuracy metrics.

Thank you.

For learner use only

Video 8 – Understanding Decision Boundaries

Welcome to this session on understanding decision boundaries in classification. Decision boundaries are crucial in understanding how classifiers separate data points into different classes. They give us a visual and conceptual sense of how a model makes decisions, whether you are working with logistic regression, SVM, or a more complex classifier.

Decision boundaries help you interpret the model's behaviour and assess its generalization capability. Let's dive deeper to see how these boundaries evolve with different models and transformations. What is a decision boundary? A decision boundary is a surface or a region in the feature space where the output class of a classifier changes.

In a two-dimensional dataset, this is typically represented as a line or a curve. For instance, in logistic regression, the boundary is linear, essentially a straight line that separates two classes. However, for models like k-nearest neighbours, the boundary can become quite irregular, adapting to the local structure of the data.

The nature of this boundary reveals how the model interprets the underlying pattern in your data. What I'm trying to tell you over here in logistic regression, it might happen that we have two features, x1 and x2, and based on that, we have some circles, which represents one type of class, and we have squares, which represents another type of class. And what logistic regression is trying to do over here is it's trying to fit a line which will act as a classifier, a straight line.

Now, the straight line might have to allow some misclassification because logistic regression is fitting a line, or it might adjust to data points and build a better model. And when you talk about models like KNNs, KNNs fit on the data, and they focus more towards perfect classification and give you this kind of decision boundary. So, depending on the type of model that you are using, your decision boundary might change.

Some linear decision boundaries, linear models such as logistic regression or your linear support vector machines produce decision boundaries that are straight lines or hyperplane in higher dimension. If you're talking about two features, two-dimensional space, we'll have a line which separates the classes. But when we have higher dimensions, meaning we have more features, more than two features, then it will start fitting a hyperplane.

These models are efficient and interpretable and work well when data is linearly separable. You can do a simple implementation using logistic regression that we have done. And we can also check how the decision boundary is coming when we are implementing a simple logistic regression model, which is what we will be looking at when we do the demo.

Then you also come across some nonlinear decision boundaries. As mentioned, you might have two features, X1 and X2. And you have some classes, circle represents one type of class, square represents another type of class.

And if you look at this data, when I want to do classification, a linear model will not work, but a nonlinear model will do a perfect classification. So you also come across scenario when data is not linearly separable. We need models that can draw curved or complex decision boundaries.

For learner use only

Models like Kernel, SVM, Decision Trees, and KNN come into play here. These models can model intricate patterns in data, making them better suited for real world applications. For example, an RBF Kernel SVM can wrap around data clusters with smooth nonlinear boundaries.

This ability to adapt to complex contour is key when dealing with messy datasets. Visualizing decision boundaries helps demystify how a model behaves. Libraries like ML-Extend, and please notice the library name is ML-Extend.

There is no E over here. And if you want to install this, you can say pip install ML-Extend. These libraries allow us to draw these boundaries, the function plot decision regions coming from ML-Extend gives a clear picture how the model divides a feature space and it will create a colour region for each class, which gives you a good intuitive understanding where one class ends and another begins.

The complexity of the model directly affects the shape of the decision boundary. A simple model like logistic regression will draw a smooth boundary, which might miss some subtle IIT Guwahati

patterns, but will generalize well. A complex model like an SVM with a high degree polynomial kernel can tightly wrap around training data points.

While this might seem accurate on the surface, it often leads to overfitting where the model memorizes noise instead of learning true patterns. Overfitting that we discussed can be understood visually through decision boundaries. To give a little perspective over here, again, if I take the example of circles and some squares, and let's say for the sake of argument, we have a circle over here.

Now, ideally, what we should do is we should ignore that one exception point and build a model like this. A linear model will work in this scenario with one wrong classification, which will be this. But most of the model, if you talk about nonlinear model, will overfit this scenario.

What they will do is they will fit a curve which will look like this and it will try to classify each training data point perfectly. And now with this new model that we have, the green one, what happens? Any point which comes in this region will be classified as circle. But because of this single data point, my decision boundary has changed and which might lead to overfitting.

So overfitting can be understood by visualizing it. The only problem is that we can visualize up to two dimensions, more than two dimensions. It becomes difficult to visualize.

But this gives you a good understanding that how these models they work. An overfitted model might show near perfect accuracy on training data, but fails miserably on unseen test data. And this is often reflected in a jagged or complex decision boundaries that try to fit every single point, including noise.

These boundaries don't generalize well and result in poor performance when deployed. In contrast, simpler boundaries that may misclassify a few training points tend to be more robust. So in this example, if you build something like this, it will give better performance on unseen data compared to the green one.

For learner use only

Different classifiers create different types of boundaries. Here is a simple comparison. Logistic regression gives a clean linear separation.

KNN with a low K value like 1 creates highly irregular and local boundaries. Increasing say K to 15 or 20 smoothens these boundaries. Decision trees produce boundaries in the form of axis-aligned rectangles, which can look like staircases.

SVM with RBF kernel creates elegant, curved boundaries. This comparative view helps in selecting the right model depending on the data structure. Feature transformation and boundary shape.

Feature transformations significantly influence the decision boundary. Taking a feature as X and then transforming to X squared or log of X will change the shape of the decision boundary. By applying transformations, we can reshape the feature space to make it linearly separable.

The transformed space allows models to draw more expressive boundaries that may appear curved in the original feature space. For instance, a logistic regression trained on polynomial

transformed features can now model nonlinear boundaries. This highlights how feature engineering and model choice go hand in hand.

Let's do one thing. Let's do a small demo here to understand. How decision boundaries do they work.

So here we are taking a simple example of building a model and then visualizing the boundaries under different conditions. We are importing libraries over here, which are required. If you notice here, we have ML extend plotting plot decision regions, which will plot the decision boundaries.

So we run this. Here we are creating some artificial data point for the sake of this particular example. We have an option in sklearn, scikit-learn, where I can say make moves that will help you to create some sample data, number of samples we are saying 200, add noise of 0.25, random state 42.

And then we need x and y component. We have splitted the data into train and test. And here is a snapshot of data.

We have x1 and x2 as feature. You can think that way, x1 and x2, first five observation and the corresponding outcome over here. So the first observation belong to class 0, second belongs to class 0 and the last two belong to class 1. OK, so this is what the data is.

And now what we are doing over here is we are building a simple logistic regression model. I'm creating a pipeline here using a pipeline argument. I'm saying sklearn, that's a name of the first component which will standardize the data.

Then we are building the logistic regression model over here. That's the second component fit on the train data. And then we are plotting the decision boundary.

This is a simple linear regression problem that we are doing. And this is how your output comes. So here is the decision boundary.

Anything above it is your circle. Anything below this is your, your, sorry, anything above this is your class 0, anything below is a class 1. So we have square and triangles over here.

For learner use only

And if you look at some of the data points over here, like, for example, this triangle over here is misclassification.

So this point is classified as zero, but the actual label is one. That's what it shows you. Similarly, you have some misclassified data points over here also.

So it gives you a good visual interpretation how the classification is working in this example. But what happens if I try with the polynomial features? Like I want to try this, I want to try from X, I want to take X cube. That's a degree of polynomial I want to try.

So now I'm building the model again. This time what I'm going to do, I'm going to choose polynomial features. Degree is equal to 3 means you are using cube of the feature and then creating new of new features from the given set of features IIT Guwahati

Then we are standardizing the data and we are fitting the logistic regression model itself. And once we fit it, we say plot decision regions. You need to provide what's your X, what's your Y value, what's the model's name, and then it plots the data.

Now look at this polynomial logistic regression model. Look at the decision boundary. It's not linear anymore.

And that's where feature transformation is so important. It's not about just given X values, take it as it is and then plot it. You need to work with your features and if required, transform it.

So now we get this kind of shape with the polynomial features. OK, so this is what we have done with the logistic regression. We can do something similar with our KNN as well.

Like we build a KNN with number of neighbours as one only. So same pipeline, very similar pipeline. It's just that the different model and here I'm starting with number of neighbours as one and I fit on the data and then I'm plotting it.

Look at the plot when I take k is equal to one. So when you take k is equal to one. Look at the plot, how it comes.

It's so intricate because it has to, it is trying to classify every data point perfectly. So you have local boundaries. But this will fail miserably if I implement in production where the new data set will come.

Because what it will do, all these data points over here. If there is a data point over here will be classified as triangle. Even if they are squares, that's where the problem would be.

And if I change the number of neighbours to 15. If I'm changing to 15, you will get to see a more relaxed boundary. A better generalized model.

So that's what number of neighbours can do to your model. And then we are trying a polynomial regression, polynomial classifier model with k nearest neighbours. So we are using polynomial features and k is equal to five.

Look at the shape. Now try and understand that how my model behaviour is changing. When I'm changing number of neighbours, when I'm doing feature transformation.

And we do not know what kind of feature transformation will work. So it is more of a trial-and-error method. You do experimentation and eventually you build very robust model.

So that's what is your decision boundaries.

For learner use only